Certificate-driven polytope-hull contraction for interval branch and bound

Ignacio Araya
Escuela de Ingeniería Informática, Pontificia Universidad Católica de Valparaíso, Chile
Victor Reyes
Escuela de Informática y Telecomunicaciones, Universidad Diego Portales, Chile

[TODO: borrador del 2026-11-04**]**

Abstract

Interval branch-and-bound solvers compute rigorous enclosures of the solutions of nonlinear systems and of global optima. Their polytope-hull contractor bounds each of the n variables of a node over a linear relaxation by solving $2n$ small linear programs (LPs) with a floating-point LP solver, whose dual solutions are certified afterwards by the Neumaier–Shcherbina bound. Since any vector of multipliers certifies a valid bound, the simplex that computes them never needs to be correct. We present a self-certifying simplex (SC) that drops the work a general-purpose LP solver spends on making its answer correct: it verifies nothing, does not refactorize before declaring an optimum, needs no phase 1, and solves the $2n$ LPs on one shared basis. A ladder of configurations in the open-source solver Ibex, from SoPlex as Ibex uses it, through the merging of twin rows, a tuned SoPlex and its simplex driven directly, to SC, attributes the gain to each step. On relaxations of up to 30 rows the redesigned contractor makes the solver about 24% faster on global optimization problems and **[TODO:** X**]**% faster on systems of equations, and SC accounts for **[TODO:** Y**]**% beyond the simplex of SoPlex driven directly. On larger relaxations SC is slower than that simplex, and a hybrid that chooses the engine by the size of the relaxation keeps the best of both. **[TODO:** cifras finales de la escalera**]**

1 Introduction

Interval branch-and-bound solvers compute, with mathematical certainty, an enclosure of all solutions of a system of nonlinear equations and inequalities, or the global optimum of a constrained problem [22]. Their efficiency depends on *contractors*, operators that shrink the current box without losing solutions [7]. Among them, contractors based on linear relaxations are among the most effective: the constraints are linearized over the box, by convex interval Taylor forms [3] or affine arithmetic [25, 2], and each variable is minimized and maximized over the resulting polytope. In Ibex this is the polytope-hull contractor, the linear part of X-Newton, which we introduced in [3]; we later combined the affine and Taylor-based relaxations [2].

Each call solves up to $2n$ linear programs, and the rigor of the result cannot come from the LP solver, which works in floating point. Current implementations therefore treat the LP solver as a black box and certify its output a posteriori: the dual solution is turned into a safe bound by the Neumaier–Shcherbina procedure [23], and an infeasibility claim is accepted only if the returned Farkas ray passes the same verification. This design has a cost that is easy to miss: the $2n$ LPs of a node are small and closely related, typically two or three pivots apart, and each is a separate

call to a general-purpose solver, which sets up its vectors, refactorizes the basis before declaring an optimum and verifies its solution; on these LPs that work exceeds the pivots themselves (Section 5).

We design the simplex around the certificate instead. With the relaxation written in a homogeneous form, with one interval-bounded slack per row, *any* multiplier vector yields a valid bound on the box and on every sub-box, so the simplex that computes the multipliers never needs to be correct: it verifies nothing and does not refactorize before declaring an optimum. Since every basis can also be made dual feasible by relocating its nonbasic variables, each of the $2n$ bound problems, and each problem after the box contracts, starts without a phase 1 (the search for a feasible starting basis), and the $2n$ problems share one basis. We call this simplex *self-certifying* (SC). How much it is worth depends on the size of the relaxation and on what else the contractor does, and measuring that is part of the contribution.

Positioning. The certificate itself is not new. That any multiplier vector yields a valid bound once the residual is evaluated in interval arithmetic is the principle of Neumaier and Shcherbina [23]. Domes and Neumaier [10] applied it to filtering with linear relaxations: they solve the bound LPs with an external solver, collect the multipliers, and solve the preconditioned interval system row by row. In previous work [26] we wrote the relaxation in the homogeneous form used here and showed that the optimal preconditioner for an interval Gauss–Seidel projection is given by the dual solutions of the bound LPs; the equivalent valid inequalities appear in the optimization-based bound tightening of Gleixner et al. [13], and both works reuse them in sub-boxes. In all these approaches the multipliers are a by-product of an external solver and certification is a post-processing step, whose cost leads to applying it to a few promising variables [10], or to recomputing it only periodically and reusing it in between [26]. What changes with SC is *where the multipliers come from and how often a usable certificate is available*: the simplex itself produces them, at every basis. On the solver side, Yamamura and coworkers [29, 30] observed that, after a bisection, the optimal tableau of the parent remains dual feasible for the child, so that an LP emptiness test can be resumed by the dual simplex method with few or no pivots per box; their warm start concerns a single feasibility LP on a fixed relaxation, without rigorous certification, and SC extends it to the $2n$ objectives of the bound problems. Bound tightening by optimization is also central in MINLP solvers [5, 13], where rigor is not required; the heuristic of skipping bounds already attained by a primal point, used by Ibex, comes from Baharev, Achterberg and Rév [4].

Contributions. The paper makes three contributions.

- *A simplex designed for the certificate.* SC omits the verification, the refactorization before an optimum and the phase 1 of a general-purpose solver, and solves the $2n$ problems of a node on one shared basis (Section 4). The properties it relies on, validity of the certificate for any multipliers, exactness at an optimum and dual feasibility for free, follow from the Neumaier–Shcherbina bound and LP duality; we state them in Section 3.1 because the design rests on them, not as new results.

- *An attribution of the gain.* A ladder of configurations inside the same contractor of Ibex, from SoPlex as Ibex uses it, through the merging of twin rows (a simple preprocessing that halves the relaxation of square systems for every engine), a tuned SoPlex and its simplex driven directly, to SC and a hybrid, measures what each step contributes, separately on optimization problems and systems of equations and on small and large relaxations, with one instance per family. A replay of recorded passes outside the search, on the same LPs, adds HiGHS and Gurobi and explains where the time of a call goes (Section 5).

- *The boundary of the approach.* SC pays on relaxations of up to about 30 rows, where the fixed cost of a call dominates; beyond, its pivots cost more than those of SoPlex. We explain why, and give a hybrid contractor that chooses the engine per node by the size of the relaxation (Sections 5 and 6).

2 Background and related work

Interval solvers and linear relaxations. Interval branch and bound alternates bisection with contraction. Ibex, on which we build, composes constraint propagation (HC4 [6]), shaving-based consistencies such as ACID [24], the interval Newton operator for square systems [21], and contractors based on linear relaxations [7]. The polytope-hull contractor computes a linear relaxation of the constraints over the box and bounds each variable over it; in X-Newton [3] the relaxation comes from convex interval Taylor forms, and the affine arithmetic relaxation of Ninin et al. [25] is an alternative which we later combined with it [2]. QUAD [19] and GloptLab [10] use linear relaxations in the same way. In previous work [26] we wrote the relaxation in the homogeneous form used here and showed that the optimal preconditioner for a Gauss–Seidel projection over it is given by the dual solutions of the bound LPs. Yamamura and coworkers [29, 30] solve an LP emptiness test at each box of a bisection search and resume it by the dual simplex method from the parent's tableau.

Rigorous bounds for linear programs. A floating-point LP solver returns approximate solutions, and two lines of work make its results rigorous. The first certifies them a posteriori. Neumaier and Shcherbina [23] showed that any approximate dual vector yields a safe bound once the residual is evaluated in interval arithmetic, and that an approximate Farkas ray proves infeasibility under the same verification; Jansson [16] gave rigorous lower and upper bounds based on the same principle, implemented in Lurupa [17]. In filtering, Domes and Neumaier [10] apply this certificate to the bound LPs of a linear relaxation, collecting the multipliers from an external solver for a few promising variables. The second line solves the LP exactly, in rational arithmetic [1], or to arbitrary precision by iterative refinement [14], as needed in exact mixed-integer programming [8]. Both treat the LP solver as given. The present work keeps the a posteriori certificate, which costs no more than one matrix–vector product, and changes the solver instead.

Optimization-based bound tightening. Minimizing and maximizing each variable over a relaxation is also a standard bound tightening technique in global optimization, where rigor is not required: range reduction in BARON [27], and optimization-based bound tightening in Couenne [5] and SCIP. Gleixner et al. [13] make it cheaper by filtering bounds already attained by feasible points, by ordering the LPs greedily, and by turning the dual solutions into valid inequalities that are propagated in the subtree. The skipping rule used by Ibex comes from Baharev, Achterberg and Rév [4]: a bound attained by a primal point found earlier needs no LP, and the next LP is warm-started from the previous one.

Simplex techniques. Our simplex uses standard ingredients of modern implementations, described in detail by Koberstein [18] for the dual method: the bounded-variable dual simplex with Harris' ratio test [15], and the revised method with a sparse LU factorization of the basis updated by Forrest and Tomlin's method [12], as in SoPlex [28]. We also tried the long-step (bound flipping) ratio test [20] and devex pricing [15], an approximation of steepest edge [11]; on the LPs of a pass

they did not pay. What is specific to our setting is not any of these techniques but their use for $2n$ closely related LPs over the same constraints, with a certificate attached to every basis.

3 The contractor

This section describes the contractor independently of the engine that solves its LPs: the relaxation and its certificate (Section 3.1), the pass over the bounds (Section 3.2) and the merging of twin rows (Section 3.3). Section 4 then describes SC, and Section 5 compares it with external LP solvers inside this same contractor. Only SC is new; the rest is the common ground on which the engines are compared.

3.1 The homogeneous relaxation and its certificate

Let $\mathbf{x} \in \mathbb{IR}^n$ be the current box. A linearizer returns a matrix $A \in \mathbb{R}^{m\times n}$ of floating-point numbers and an interval vector $\mathbf{b} \in \mathbb{IR}^m$, possibly with infinite bounds, such that

$$x \in \mathbf{x} \text{ is a solution } \implies Ax \in \mathbf{b}. \tag{1}$$

We introduce one slack per row, $s = Ax$, and write the relaxation in the homogeneous form

$$\bar{A}z = 0, \qquad z = (x, s) \in \mathbf{z} := \mathbf{x} \times \mathbf{s}, \qquad \bar{A} := [\, A \;\; -I_m \,], \tag{2}$$

where $\mathbf{s}_i := \mathbf{b}_i \cap \square(A_{i:}\,\mathbf{x})$ and $\square(\cdot)$ denotes evaluation in interval arithmetic. We call $P(\mathbf{x}) := \{z \in \mathbf{z} : \bar{A}z = 0\}$ the *relaxation polytope*.

Lemma 1. *If $x^* \in \mathbf{x}$ is a solution, then $(x^*, Ax^*) \in P(\mathbf{x})$.*

Proof. $A_{i:}x^* \in \mathbf{b}_i$ by (1) and $A_{i:}x^* \in \square(A_{i:}\mathbf{x})$ by the inclusion property of interval arithmetic; the equation holds by construction of s. □

The certificate combines the m rows of $\bar{A}z = 0$ linearly, with one *multiplier* per row, into a single equation $\gamma^T z = 0$ that every point of the polytope satisfies. Any vector of multipliers $\lambda \in \mathbb{R}^m$ gives a valid equation; the best ones for bounding z_k are the dual solutions of the LP that minimizes or maximizes z_k over the polytope, and the simplex of Section 4 produces them.

Definition 2 (Certificate)**.** For a vector of multipliers $\lambda \in \mathbb{R}^m$, let $\gamma \in \mathbb{IR}^{n+m}$ be an interval vector that contains $\gamma := \bar{A}^T\lambda$, computed in outward-rounded interval arithmetic. For $k \in \{1, \ldots, n+m\}$ with $0 \notin \gamma_k$, the *certified enclosure* of z_k is

$$\mathbf{C}_k(\lambda, \mathbf{z}) := -\Big(\textstyle\sum_{j\neq k} \gamma_j\, \mathbf{z}_j\Big)\Big/\gamma_k,$$

and the *residual* is $\mathbf{R}(\lambda, \mathbf{z}) := \sum_j \gamma_j\, \mathbf{z}_j$.

The idea is a single line. Every point z of the polytope satisfies $\bar{A}z = 0$, hence $\gamma^T z = \lambda^T \bar{A} z = 0$ for any λ, that is, $\gamma_k z_k = -\sum_{j\neq k} \gamma_j z_j$. Bounding the right-hand side over the box in interval arithmetic, and dividing by γ_k, encloses z_k; and if the residual $\mathbf{R}$ excludes zero, no point of the box can satisfy $\gamma^T z = 0$, so the polytope is empty. A contraction step therefore replaces $\mathbf{z}_k$ by $\mathbf{z}_k \cap \mathbf{C}_k(\lambda, \mathbf{z})$, and an emptiness test checks $0 \notin \mathbf{R}(\lambda, \mathbf{z})$. All statements below hold for the floating-point A returned by the linearizer; rounding in γ and $\mathbf{C}_k$ is covered by outward rounding.

Proposition 3 (Validity). *Let $\lambda \in \mathbb{R}^m$ be arbitrary. For every $z \in P(\mathbf{x})$ and every k with $0 \notin \gamma_k$, $z_k \in \mathbf{C}_k(\lambda, \mathbf{z})$; if $0 \notin \mathbf{R}(\lambda, \mathbf{z})$, then $P(\mathbf{x}) = \emptyset$ and $\mathbf{x}$ contains no solution. Both claims hold on every sub-box $\mathbf{x}' \subseteq \mathbf{x}$ with the matrix $\bar{A}$ computed on $\mathbf{x}$.*

Proof. For $z \in P(\mathbf{x})$ and the exact $\gamma = \bar{A}^T\lambda \in \gamma$, $\gamma^T z = \lambda^T \bar{A} z = 0$; both claims follow from the inclusion property of interval arithmetic and Lemma 1. The linearization on $\mathbf{x}$ remains valid on $\mathbf{x}'$, so the polytope over $\mathbf{x}'$ lies in $P(\mathbf{x})$. □

No property of λ is used: it need not be optimal, feasible or accurate, and errors in it can only weaken the enclosure. Written back in the variables x, $\inf \mathbf{C}_k(\lambda, \mathbf{z})$ is the Neumaier–Shcherbina safe lower bound of $\min x_k$ subject to $Ax \in \mathbf{b}$, $x \in \mathbf{x}$, with approximate dual λ [23], and the row-wise solution of the preconditioned system of Domes and Neumaier [10, §5]; validity on sub-boxes is used by Gleixner et al. [13] and in our earlier work [26]. The homogeneous form, introduced in [26], adds one detail: each slack is bounded by $\square(A_{i:}\mathbf{x})$ even when a side of $\mathbf{b}_i$ is infinite, so a multiplier of the "wrong" sign on a one-sided row gives a weaker but finite bound, and literally any λ certifies. Two standard facts complete what the design uses. At an optimal basis of LP_k with z_k basic, $\inf \mathbf{C}_k(\lambda^*, \mathbf{z})$ equals the optimal value in exact arithmetic, by LP duality [26, Prop. 1]; in floating point the excess is at most $\sum_j \mathrm{wid}(\gamma_j)\,\mathrm{mag}(\mathbf{z}_j)$ plus the effect of the inaccuracy of λ, as for the Neumaier–Shcherbina bound. And every basis can be made dual feasible for free [18]:

Proposition 4 (Dual-feasible start). *The slack basis $B = -I_m$ is dual feasible for the bound problem of every structural variable z_k, $k \le n$, with $\inf \mathbf{z}_k$ finite. If every $\mathbf{z}_j$ is bounded, any basis becomes dual feasible after placing each nonbasic z_j at its lower bound if its reduced cost $d_j > 0$ and at its upper bound if $d_j < 0$.*

Proof. With the slack basis the cost of every basic variable is zero, so $\lambda = 0$ and $d = e_k$, and z_k at its lower bound has the right sign. In general d does not depend on where the nonbasic variables sit, and the relocation enforces the sign conditions whenever the required bound is finite. □

When the objective changes from one bound to the next, or the box contracts, relocation thus restores dual feasibility without a phase 1, which is what lets one basis serve the $2n$ problems. Yamamura and coworkers [29, 30] used the case of a box change to resume an LP emptiness test after a bisection. Section 4 explains how the simplex produces λ.

Why not a rigorous simplex. Validity does not depend on how λ was computed: any vector of multipliers gives a valid bound, and a poor one only gives a weaker bound. Rigor is therefore needed only in the final evaluation of γ and $\mathbf{C}_k$, not in the pivoting. Carrying intervals through the pivots instead pays the overestimation of interval Gaussian elimination [21], whose widths grow with the number of pivots, and solving each LP exactly in rational arithmetic, or to high precision by iterative refinement [1, 14], buys an exactness that the bound does not need, at a cost that $2n$ LPs per node cannot afford. An early version of our contractor pivoted in interval arithmetic: on a one-box benchmark it had to rebuild the tableau from the original relaxation 731 times to control the width growth, and pivoting in floating point with certification of the final multipliers only was 2.8 times faster with essentially the same contraction (mean 24.0% against 24.6%).

3.2 One pass over the bounds

At each node the relaxation is computed, written in the form (2), and loaded once. The contractor then makes one pass over the up to $2n$ bound problems

$$\mathrm{LP}_k: \quad \min\ z_k \quad \text{s.t.} \quad \bar{A}z = 0,\ z \in \mathbf{z}, \tag{3}$$

and their maximizations, obtained with $-z_k$, each LP warm-started from the final basis of the previous one. When an LP ends, its bound is certified once, from the multipliers of the final basis, and the box is contracted before the next LP starts. Two rules decide which LPs are solved and in which order. A bound is *skipped* when a primal point obtained earlier in the pass already attains it, since its LP would return that value; this rule comes from the polytope-hull contractor of Ibex [4]. Among the remaining bounds, the contractor of Ibex solves first the one farthest from the last primal point; we solve first the nearest one, whose optimum lies closest to the vertex where the previous LP ended, so that the next LP needs fewer pivots. This order is the best one for every engine we compare.

If the residual $\mathbf{R}(\lambda, \mathbf{z})$ of an LP excludes zero, the node is empty and the pass stops. No other outcome stops it. An LP that ends without an optimum, at its iteration limit, still has the multipliers of the basis where it stopped, which certify a weaker but valid bound (Proposition 3); an LP that reports an infeasibility whose ray fails the residual test leaves its bound uncontracted, and is solved once more from a cold start. The polytope-hull contractor of Ibex 2.8.9 instead abandons the pass in both cases; we corrected it, the correction has been submitted to the Ibex distribution, and every engine of Section 5 uses it, including the reference. Ibex limits SoPlex to 100 iterations per LP; the other configurations, SC included, use $20m + 200$, where m is the number of rows.

3.3 Merging twin rows

Rows of the relaxation that are proportional, within a relative tolerance of 10^{-9}, are merged into one row whose slack interval is the intersection of their scaled intervals; the polytope does not change. The affine linearizer writes each equation as two one-sided rows with the same normal, so merging halves m on square systems, and the combined relaxation of Section 5 writes some rows twice, with opposite signs, far apart in the matrix. Merging is done once per node, before any LP, and gives a smaller LP to SC and to an external solver alike.

4 A simplex for the $2n$ bound problems

For each structural variable x_k the pass of Section 3.2 needs the two bounds of LP_k (3). A basis is a set of m columns of $\bar{A}$ forming a nonsingular matrix B; with cost vector $c = \pm e_k$, the multipliers are $\lambda^T = c_B^T B^{-1}$ and the reduced costs $d = c - \bar{A}^T\lambda$. For the bounded-variable simplex, a basis with each nonbasic variable at one of its bounds is *dual feasible* if $d_j \geq 0$ for nonbasic variables at their lower bound and $d_j \leq 0$ for those at their upper bound, and *primal feasible* if the basic variables lie within their bounds. This section describes how the simplex solves the $2n$ problems of a node; the design choices are evaluated in Section 5.

What the certificate makes unnecessary. Since the certificate of Section 3.1 is valid for any λ (Proposition 3), the simplex never has to return a correct solution, only a good basis. A general-purpose LP solver, before declaring an optimum, refactorizes the basis to remove the error accumulated by the updates of its factorization, recomputes its vectors and verifies the solution, and the contractor then verifies it once more through the certificate. SC uses the final basis as it is: an inaccurate λ yields a weaker bound, never an invalid one, so the error of the factorization is controlled only as far as pivoting requires. Likewise, since every basis can be made dual feasible (Proposition 4), no phase 1 is needed when the objective changes or the box contracts, and the $2n$ problems can share one basis. Section 5.2 measures what these savings are worth against SoPlex.

The rest of this section describes the simplex that results: how each LP starts from the previous one (Section 4.1), the iterations (Section 4.2), the factorization of the basis (Section 4.3), and how the certificate is computed when an LP ends (Section 4.4).

4.1 Starting and warm-starting

The first LP of a pass has no previous basis. It starts from the slack basis $B = -I_m$, which is dual feasible for every LP_k (Proposition 4), and is solved by the dual method, with no phase 1.

Between two consecutive LPs of the pass the objective changes, and the contracted bound changes the box. Two warm starts are possible from the previous optimal basis:

- *dual*: relocate every nonbasic variable to the bound indicated by the sign of its new reduced cost, z_k included, which restores dual feasibility (Proposition 4), and continue with the dual method; the certified bound is then available, and nondecreasing, at every iteration (Proposition 5);
- *primal*: keep the nonbasic variables where they are; the basis remains primal feasible, up to the contracted bound, and is reoptimized with primal steps, which is the usual warm start of an LP solver.

Under the dual method the certificate is available at every iteration, and it never gets worse:

Proposition 5 (Monotone certified bound)**.** *Run the dual simplex method on* LP_k *from a dual-feasible basis. At every iteration* t *in which* z_k *is basic,* $\inf \mathbf{C}_k(\lambda_t, \mathbf{z})$ *is a valid lower bound on* z_k *over* $P(\mathbf{x})$ *and, in exact arithmetic, it is nondecreasing in* t*.*

Proof. Validity is Proposition 3. With z_k basic, $d = e_k - \bar{A}^T\lambda_t$ has $d_k = 0$, so $\gamma_k = 1$, $\gamma_j = -d_j$ for $j \neq k$, and $\inf \mathbf{C}_k = \sum_{j\neq k} \min_{z_j \in \mathbf{z}_j} d_j z_j = \min_{z\in\mathbf{z}} d^T z$, the Lagrangian dual function of LP_k at λ_t, which the iterates of the dual simplex method do not decrease. □

An LP stopped at its iteration limit under the dual method therefore keeps a bound no weaker than at its start (Section 3.2). Both reach an optimal basis, so the certificate at termination is exact (Section 3.1) and the pruning is the same; whether the node is empty is decided in the first LP, common to both. They differ only in cost.

The cost depends on how far apart the optima of consecutive bounds are. An optimal vertex is given by a basis and by the bound on which each nonbasic variable rests, and a nonbasic variable that must move to its opposite bound costs the two warm starts differently: the primal one pays an iteration for it, the dual one moves it for free, since relocation is not a pivot. Relocation has its own cost, however, paid once per LP: it changes the values of the nonbasic variables, and the basic values must be recomputed from the factorization. So the primal warm start is cheaper when few variables change side, and the dual one when many do.

How many change side depends on the rows. On systems of equations every row comes from an equation and is a narrow slab: the interval of its slack is, on average, 13% of the range of the row over the box, so the optima of consecutive bounds often rest on opposite faces of a slab. On optimization problems the slack intervals cover 48% of the range, and few rows change side. Table 1 measures it over the benchmarks of Section 5. The basis changes by 10–17% in both programs, but a median of 1.9 nonbasic variables per LP change side on systems of equations, against 0.3 on optimization problems. The iterations follow: on systems the primal warm start makes 5.1 per LP against 3.1 for the dual one, about one more per side change; on optimization problems it makes 2.1 against 2.5.

Table 1: Optima of consecutive bounds of a pass (seed 1, 60 s per run, 165 systems and 141 optimization problems with at least 1 000 warm LPs in both warm starts). Medians, with quartiles in parentheses, of the rows of the relaxation, the basic variables that change, the nonbasic variables that move to their opposite bound, and the iterations per warm LP with each warm start.

		basis	side changes		iterations per LP	
benchmark	rows	changed	per LP	% nonbasic	dual	primal
systems	18 (10–32)	10%	1.9 (1.1–2.8)	16%	3.1	5.1
optimization	9 (5–17)	17%	0.3 (0.2–0.5)	4%	2.5	2.1

In time, the two effects balance differently in each program. On optimization problems the primal warm start pivots as little as the dual one and saves the relocation: in the whole solver, the dual warm start is 4% slower (Table 5). On systems of equations the two extra primal pivots per LP cost about as much as the relocation they avoid, and both warm starts take the same time. SC therefore uses the primal warm start in the optimizer, where the first LP of each pass is still solved by the dual method, and the dual warm start in the solver of systems of equations. There the two cost the same and the choice matters little; we keep the dual one so that the whole pass runs the dual method of its first LP, with primal steps only as a fallback. With the larger relaxation of Section 5 neither warm start wins on every system, and SC chooses per run: it alternates the two on the first nodes, keeps the one with the lower cost per LP, measured as a moving average, and tries the other again every 25 nodes. With the primal warm start, primal steps are also the fallback when relocation is impossible because the bound indicated by a reduced cost is infinite; with the dual one, SC takes no primal steps, and an LP that ends with such a variable gives a valid, possibly weaker bound.

4.2 Simplex iterations

We recall the bounded-variable simplex in the notation of LP_k (3), following [18]. Let B be the basis matrix, $\mathcal{B}$ and $\mathcal{N}$ the basic and nonbasic index sets, and x_N the nonbasic variables, each at one of its bounds; the basic values are $x_B = -B^{-1}\bar{A}_{\mathcal{N}} x_N$, since the right-hand side of (2) is zero. Every iteration exchanges one basic and one nonbasic variable and needs three vectors:

- the *pivot column* $\alpha_q = B^{-1}\bar{A}_{:q}$ of the entering variable z_q, obtained by solving $B\alpha_q = \bar{A}_{:q}$ (forward transformation, FTRAN);
- the *pivot row* $\alpha_r^T = e_r^T B^{-1}\bar{A}$ of the variable leaving at position r, obtained by solving $B^T\rho_r = e_r$ (backward transformation, BTRAN) and then computing $\alpha_r^T = \rho_r^T\bar{A}$;
- the step lengths: a primal step θ_p for x_B and a dual step $\theta_d = d_q/\alpha_{rq}$ for (λ, d).

The updates are then

$$x_B \leftarrow x_B - \theta_p\,\alpha_q, \qquad \lambda \leftarrow \lambda + \theta_d\,\rho_r, \qquad d \leftarrow d - \theta_d\,\alpha_r,$$

with z_q taking position r of the basis. The certificate needs λ, which is therefore always at hand.

Dual steps. In a dual iteration the basis is dual feasible and some basic variable violates its bounds. The leaving variable is the basic variable with the largest violation. The entering variable

comes from the ratio test over the pivot row: among the nonbasic z_j whose reduced cost would change sign in the wrong direction, the one with the smallest $|d_j/\alpha_{rj}|$ keeps the basis dual feasible; among the candidates within a relative band of that ratio we take the largest $|\alpha_{rj}|$ (Harris' rule), which avoids tiny pivots.

Rules that did not pay. We also implemented the standard refinements of the dual method [18]: devex pricing [15] for the leaving variable, the long-step (bound flipping) ratio test [20], and a limit of 60 pivots per LP, which the certificate makes safe (Proposition 5). Each helps on some families and hurts on others: together they make the Bratu systems 1.3–1.5 times faster and the BroydenBanded ones up to 2.6 times slower, and on average they leave the time unchanged (Section 5). SC therefore uses the textbook rules above. On the small LPs of a pass, the choice of pivots matters less than the work that surrounds them.

Primal steps. In a primal iteration the basis is primal feasible and some nonbasic variable has a reduced cost of the wrong sign; it enters (Dantzig's rule: largest $|d_j|$), and the ratio test over the pivot column α_q finds the first basic variable to reach a bound, or lets z_q move to its own opposite bound. Pivots below a stability threshold are filtered out before choosing the leaving variable; without this filter a rejected pivot was indistinguishable from optimality, and 40% of the reported optima were not dual feasible. SC takes primal steps only with the primal warm start: to reoptimize after each bound, and to repair the nonbasic variables that could not be relocated.

Tolerances and infinite bounds. The primal and dual feasibility tolerances are 10^{-9}, relative to the magnitude of the value tested; with the combined relaxation of Section 5, whose nearly parallel rows make the LPs degenerate, we use 10^{-6} in the solver of systems of equations. A tolerance is a matter of efficiency, not of rigor. A tolerance too wide accepts an empty polytope as feasible, which only delays the pruning, and weakens the contraction: with 10^{-6} and the affine relaxation, SC explores twice the cells on Geneig-icse. A tolerance too tight makes the warm-started simplex report infeasibility on boxes that are not empty; the ray then fails the residual test, and SC solves that LP again from the slack basis instead of discarding it. Slacks may have an infinite side; a nonbasic variable always sits at a finite bound, the other one if the bound indicated by its reduced cost is infinite, and at zero if it is free.

4.3 The engine

SC keeps a sparse LU factorization $B = LU$, computed by the factorization code of SoPlex, and obtains each iteration's vectors by triangular solves: FTRAN solves $Ly = \bar{A}_{:q}$ and then $U\alpha_q = y$, BTRAN solves $U^T w = e_r$ and then $L^T \rho_r = w$. Since the basis changes by one column per iteration, the factors are not recomputed but updated: in the Forrest–Tomlin update [12] the new column replaces column r of U, which breaks its triangular shape; a permutation moves it to the end, and one row elimination, stored as a sparse row transformation appended to L, restores the triangle. The update reuses the vector y of the last FTRAN, so it costs about one more solve. Updates make the factors grow, and SC refactorizes B from scratch every 50 pivots. The FTRAN of the entering column is computed once per iteration and serves both the Forrest–Tomlin update and the update of x_B, which is recomputed from scratch every 32 pivots to limit drift. The reduced costs and λ are updated with the pivot row as above. An iteration costs one FTRAN, one BTRAN, one product $\rho_r^T \bar{A}$ and the update, all proportional to the number of nonzeros involved.

4.4 Certification

When an LP terminates, λ is read from the final basis, where it is kept updated with the pivot row. The certificate $\gamma = \bar{A}^T\lambda$ is computed once per bound in outward-rounded interval arithmetic, over the rows with $\lambda_i \neq 0$, and $\mathbf{C}_k$ and $\mathbf{R}$ follow in $O(n+m)$ (Definition 2). When a dual iteration finds a basic variable out of its bounds and no entering variable in its row r, the polytope is empty, and the row r of B^{-1} is a Farkas ray: SC takes $\lambda = \rho_r$, and the residual test turns it into a proof of emptiness. The simplex itself never needs to be correct: a wrong pivot or an inaccurate λ yields a weaker bound, never an invalid one (Proposition 3). Certifying a bound costs the same as the Neumaier–Shcherbina post-processing of an external solver's dual, since both compute $\bar{A}^T\lambda$ over the rows with $\lambda_i \neq 0$ plus $O(n+m)$ operations: 1.3–3.1 μs per LP on three optimization problems, against 1.9–3.0 μs plus 1.2–1.5 μs to extract the solution from the external solver. The difference between the approaches is not the cost of a certificate but where λ comes from.

5 Experimental evaluation

A controlled comparison. To measure the contribution of the simplex, and not of the rest of the contractor, we compare engines inside the *same* contractor. All run the same pass over the $2n$ bounds, on the same merged relaxation, with the same skipping rule, the same certificate and the same policy when an LP yields no certificate: that bound is not contracted and the pass continues. (The polytope-hull contractor of Ibex 2.8.9 abandons the pass when the LP solver reports infeasibility without a verifiable ray or reaches its iteration limit; our contractor continues with the next bound, whatever the engine.) They differ only in where the multipliers come from:

- *SC*: the multipliers of the final basis of the self-certifying simplex;
- a *black box*: an LP solver solves each LP on the same relaxation, warm-started from the previous bound, and its dual solution, or its Farkas ray if it reports infeasibility, is passed to the same interval certificate $\gamma = \bar{A}^T\lambda$, which is the Neumaier–Shcherbina bound written in the homogeneous form.

The homogeneous form is therefore not what separates the engines: the certificate, its evaluation over the box and its validity on sub-boxes are common to all, and SoPlex itself solves ranged rows in the same form, with one bounded logical variable per row. What differs is a simplex designed for that certificate.

We use four black boxes. The reference is SoPlex 4.0.2 driven as Ibex drives it, through its `SoPlex` class, in its best configuration for this use: its own instance holding the same rows and sides with the box as column bounds, presolving off, the nearest bound first, and the tolerance 10^{-6} for systems of equations and 10^{-9} for optimization.[1] The second is the simplex of SoPlex driven directly through its `SPxSolver` class, the *bare simplex*: the LP stays loaded, between bounds only the objective and the column bounds change, and nothing is verified. It is the cheapest way to use SoPlex, and we report it alongside SoPlex throughout. The other two are HiGHS 1.15.1 and Gurobi 12.0.3, with one thread, the dual simplex, the basis kept between LPs and no presolve; a

[1] The LP interface of Ibex also stores the n variable bounds as rows, which makes each LP 16% more expensive on small relaxations; rows that are free for SoPlex are passed to it empty; presolving each new relaxation costs 3% more than it saves on LPs of a few pivots; the farthest bound first is 3–7% slower; 10^{-9} makes SoPlex report spurious infeasibility on relaxations with nearly equal rows, and 10^{-6} weakens its dual on optimization problems. Turning off scaling, or forcing the row or column representation or the primal algorithm, did not help, and SoPlex 7.1.6 was 1.2–1.9 times slower than 4.0.2 on a sample of eight problems.

pass with them costs two to five times that of SoPlex (Table 2), so we use them only on the bank of passes. Both forms of SoPlex have the iteration limit $20m + 200$ (Section 4).

Setting. Contractor and solvers are implemented in Ibex, with the two relaxations of our previous work [2]: the affine relaxation [25], and the combined relaxation (`compo`), which adds to the affine rows those of X-Newton [3], one per constraint and corner. Preliminary experiments fixed one relaxation per program: the affine relaxation for optimization, where `compo` costs more than it saves with every engine, and `compo` for systems of equations, where it halves the search tree. Every engine solves the same relaxation. Per node, the affine relaxation has a median of 5 rows (quartiles 3–9) on optimization problems and 16 (9–30) on systems of equations after merging; `compo` has 13 (7–20) and 56 (36–94). For systems of equations we use the solver composition of Ibex (HC4, ACID, interval Newton when the system is square, and the linear contractor within a fixed point with HC4) with the LSmear bisector, on the 242 systems of the Ibex solver benchmark that some configuration solves within 3600 s (22 are excluded), 2 seeds, 60 s per run. For optimization we use IbexOpt on 240 problems, those of the Ibex optimization benchmark that some configuration solves within 3600 s (59 are excluded), 4 seeds, 60 s per run. The parameters of SC were chosen on the systems and on 139 of the optimization problems, except the bound order of the optimizer, chosen on all 240; on the other 101, which we added afterwards, SC gains as much as on the 139. Three problems (alkyl, alkylbis, bearing) whose search trees change by two orders of magnitude between seeds of the same configuration are excluded from the means. We report geometric means of ratios over the pairs of runs in which both methods finish, for runs whose reference time exceeds 0.1, 1 and 5 s, and the runs that finish within the time limit (for optimization, the problems solved in all four seeds). All runs were made on a machine with two Intel Xeon E5-2620 v4 processors (2.1 GHz, 16 cores) and 64 GB of memory, under Linux 6.17, eight runs at a time and a memory limit of 6 GB per run; the code is compiled with GCC 13.3 (`-O3`) against Ibex 2.8.9 with the GAOL 4.2.0 interval library and SoPlex 4.0.2. CPU times are process times. **[TODO:** publicar el codigo y los scripts]

5.1 Rigor

SC is rigorous by Proposition 3; we also checked it against the LP hull. On four systems (Bratu-0060, Discrete-Integralf2-13, Geneig, Bellido), for each of the 2 834 bounds where SC contracted more than the polytope-hull contractor of Ibex on the same box and polytope, we solved the bound LP over the input box with a fresh instance of SoPlex, used as an oracle; no valid contraction can go beyond this hull. SC attained it in 98% of these bounds (the contractor of Ibex in 4%) and went beyond it in 19, all within the 10^{-9} feasibility tolerance of the oracle.

5.2 Cost of a pass

The comparison inside the search mixes two effects: how many cells are explored and what each pass of the contractor costs. To measure the polytope-hull contractor alone, without the noise of the search, we built a bank of passes: for 71 instances, one per family on systems of equations and 35 optimization problems spread over the range of running times, we recorded the input boxes of 1 000 calls of the contractor along the search of SC, with the affine relaxation, and replayed each call outside the search with each engine on the same LPs. All engines then contract every box to the same result, and only their cost differs (Table 2). A pass of SC costs 0.62 times that of SoPlex on systems of equations and 0.55 times on optimization problems, and it is cheaper on all but one instance. Against the bare simplex the ratios are 0.82 and 0.73, and SC is cheaper on 26 of the 36

Table 2: Polytope-hull passes replayed outside the search (1 000 recorded calls per instance, 36 systems of equations and 35 optimization problems, affine relaxation): time per pass of SC over that of each black box on the same LPs (geometric mean), median time per pass of the black box in μs, and instances where SC is cheaper. The median pass of SC takes 245 μs on systems and 107 μs on optimization problems. All engines empty the same boxes and contract the perimeter by 0.164–0.166 on systems and 0.273 on optimization problems. HiGHS failed on one optimization problem.

	systems			optimization		
black box	SC/BB	median	cheaper	SC/BB	median	cheaper
SoPlex, as used by Ibex	0.62	373	35 / 36	0.55	196	35 / 35
SoPlex, bare simplex	0.82	280	26 / 36	0.73	140	32 / 35
Gurobi 12.0.3	0.27	892	36 / 36	0.21	576	35 / 35
HiGHS 1.15.1	0.19	1 402	36 / 36	0.12	998	34 / 34

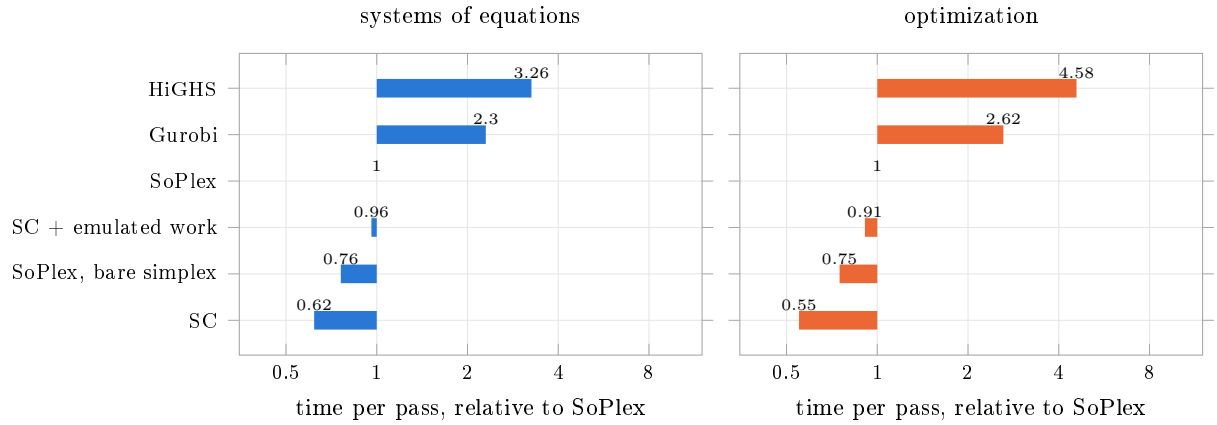


Figure 1: Time per pass on the bank of Table 2, relative to SoPlex as used by Ibex (geometric means over the instances). "SC + emulated work" is SC with the work of a call of SoPlex added at every LP (see text).

systems and on 32 of the 35 optimization problems. Against HiGHS and Gurobi a pass of SC costs 0.12–0.27 times theirs. The difference lies in the cost of an LP, not in the pivots: SC makes 28% more pivots per LP than SoPlex on systems of equations and 59% more on optimization problems, where SoPlex ends many LPs at a vertex that is already optimal for the next bound (Section 6).

Where the gain comes from. To attribute the gap with SoPlex we added to SC, at every LP and on separate vectors so that its contraction does not change, each part of the work that SoPlex does in a call, and replayed the bank.[2] In logarithmic scale the shares add up: a refactorization at every LP accounts for 53–58% of the gap, verifying the solution for 21–23%, and recomputing the vectors at the start of a solve and copying the solution for 10–11%. With all of them a pass of SC costs 0.91–0.96 times that of SoPlex (Figure 1); the rest is the machinery of a call. Driving the simplex of SoPlex directly, which keeps the LP loaded and verifies nothing, removes about half of the same gap. About half of the difference with SoPlex as used by Ibex is therefore its interface,

[2]Measured with an earlier version of SC, whose pass cost the same, 0.62 and 0.55 times that of SoPlex.

Table 3: SC against SoPlex as used by Ibex (BB) and against the bare simplex, inside the same contractor, with the affine and the combined (`compo`) relaxation. Ratios of cells (range over the three thresholds) and CPU for runs whose reference time exceeds 0.1, 1 and 5 s, and runs finished (optimization: problems solved in all four seeds, out of 240; systems: runs out of 484).

			CPU			
benchmark	comparison	cells	> 0.1 s	> 1 s	> 5 s	finished
optimization, affine	SC / BB	0.98–0.99	0.83	0.83	0.84	217 / 214
optimization, affine	SC / bare	0.98–0.99	0.90	0.90	0.90	217 / 216
systems, affine	SC / BB	0.99	0.92	0.91	0.91	365 / 362
systems, affine	SC / bare	0.98–0.99	0.96	0.96	0.94	365 / 362
optimization, compo	SC / BB	0.99–1.00	0.92	0.94	0.95	219 / 216
optimization, compo	SC / bare	0.99–1.00	1.00	1.02	1.02	219 / 218
systems, compo	SC / BB	1.00–1.01	0.99	0.99	0.98	366 / 367
systems, compo	SC / bare	1.01–1.03	1.07	1.08	1.07	366 / 372

and the other half is the refactorization that its simplex makes before declaring an optimum, which the certificate makes unnecessary.

5.3 The whole solver

Table 3 compares SC with SoPlex and with the bare simplex inside the search, with both relaxations. The search trees are the same within 3% in every comparison; what changes is the time.

With the affine relaxation, SC is 16–17% faster than SoPlex on optimization problems and 8–9% faster on systems of equations, and 10% and 4–6% faster than the bare simplex; it is faster than SoPlex in 93% of the runs on systems of equations and 92% on optimization problems. The pass is only part of the cost of a node, with the linearization, propagation, ACID and interval Newton, so the 38–45% saved per pass yields these 8–17%. The largest gains are on Geneig-icse ($\times 0.54$ against SoPlex) and DiscreteBoundary-0200 ($\times 0.69$), and on hydro, hs100 and ex2_1_9 ($\times 0.58$–0.69). Against SoPlex, SC loses by more than 5% on no system and on two optimization problems (linear $\times 1.11$, haldmads $\times 1.25$).

With `compo`, which has two to four times the rows, the picture changes. On optimization problems SC is still 5–8% faster than SoPlex, but ties with the bare simplex; on systems of equations it ties with SoPlex and is 7–8% slower than the bare simplex. The larger relaxation prunes more, for every engine: it halves the cells on systems of equations and saves 24–35% of them on optimization problems, but its LPs fall where SC loses (Section 5.4).

Where the black box loses bounds. With the same certificate, the engines can only differ in their multipliers. Running SC and SoPlex on the same boxes and polytope at every node of four systems shows two mechanisms. When SoPlex ends without an optimal basis the bound is lost, while an LP of SC that stops early yields a weaker bound, not a lost one; with the limit that Ibex gives SoPlex (100 iterations) this happens in 377 of 61 815 LPs on Geneig-icse, and with $20m + 200$ in 8 of 68 757. When both reach an optimum, degenerate LPs have many optimal duals and each engine stops at a different one. These differences go both ways (on Geneig-icse SoPlex contracts more in 40 of 1 934 nodes and SC in 32; on CountercurrentReactors2-9 SC in 46 of 1 055 and SoPlex in 22), and the mean contraction per node is the same within 10^{-3} on three of the four systems.

Table 4: CPU of SC over that of SoPlex (BB) and of the bare simplex, by the mean number of rows of the relaxation per node, for the instances whose reference runs exceed 1 s (geometric mean over the instances; number of instances in parentheses; "–": no instance).

		rows of the relaxation				
comparison	benchmark	< 15	15–30	30–60	60–120	> 120
SC / BB	optimization, affine	0.83 (52)	0.86 (15)	0.88 (2)	–	–
	optimization, compo	0.87 (26)	0.91 (19)	1.05 (15)	1.06 (8)	–
	systems, affine	0.91 (40)	0.91 (23)	0.90 (5)	0.96 (8)	0.89 (7)
	systems, compo	0.88 (3)	0.94 (9)	0.99 (35)	0.95 (25)	1.17 (10)
SC / bare	optimization, affine	0.87 (52)	1.00 (14)	0.86 (2)	–	–
	optimization, compo	0.90 (28)	1.04 (21)	1.16 (15)	1.10 (8)	–
	systems, affine	0.95 (40)	0.94 (23)	0.95 (5)	1.07 (8)	0.97 (7)
	systems, compo	0.92 (3)	1.04 (7)	1.11 (34)	1.03 (25)	1.13 (10)

Long runs. We reran with a limit of 3600 s and the affine relaxation the instances on which some configuration reached the 60 s limit (48 systems with 2 seeds, 24 optimization problems with 4 seeds); all engines re-solve from a cold start an LP whose reported infeasibility cannot be certified. On optimization problems the gain carries over: SC is 15% faster than SoPlex and 5% faster than the bare simplex, and finishes all 96 runs, against 88 for SoPlex (hs101 and hs112 unfinished in every seed) and 95 for the bare simplex. On systems of equations, where the long runs are those of the largest relaxations, SC is 4% faster than SoPlex and ties with the bare simplex; it finishes 93 of 96 runs against 94. The run that SC leaves unfinished and the others finish, of CountercurrentReactors2-26, comes from the adaptive tuning of ACID, which in some runs settles for a long stretch on shaving few variables, not from the linear contractor.

5.4 The gain depends on the size of the relaxation

Table 4 groups the instances by the mean number of rows of their relaxation, after merging. Against SoPlex, SC is faster up to 30 rows with both relaxations and in every size with the affine one; with `compo` it is within 6% from 30 to 120 rows and loses beyond (systems, $\times 1.17$). Against the bare simplex, SC wins only below 15 rows ($\times 0.87$–0.95); from 15 rows on it ties with the affine relaxation and loses with `compo` ($\times 1.03$–1.16). The affine relaxation of the optimization problems almost never exceeds 30 rows, which is why the gain of SC on them is the largest; that of the systems of equations spreads over all sizes, and `compo` moves both benchmarks to the sizes where SC loses.

On small relaxations an LP needs one to three pivots and the fixed cost of a call dominates: that is what SC removes. On large ones the pivots dominate, and SC needs more of them. On Katsurabis-14 with `compo`, the clearest case, a pass of SC costs 2.7 times that of SoPlex on the same LPs, a factor that splits into three that multiply. The starting point, $\times 1.5$: started from the final basis of SoPlex instead of its own, SC drops from 33 to 22 pivots per LP, because on a large degenerate polytope the optimum of a bound is a face, and SoPlex stops inside it at vertices that serve the next bound better. The pivoting rules, $\times 1.25$: 22 against 17.7 pivots from the same basis. The cost per pivot, $\times 1.4$ (19.4 against 14.0 μs), mostly the dual ratio test over the whole pivot row and recomputing x_B, which SoPlex updates. The limit of 60 pivots per LP cuts the longest chains and is worth 3–9% on systems of equations with `compo`, but does not close the gap.

Table 5: Ablation, affine relaxation: each row changes one component of SC. Ratios against full SC: cells and CPU on systems of equations for runs above 0.1, 1 and 5 s, runs finished out of 484 (full SC: 365), CPU on optimization problems above 0.1 s and problems solved in all seeds (full SC: 217). The warm-start row gives each program the warm start of the other: primal on systems of equations, dual on optimization problems.

	systems of equations					optimization	
configuration	cells	> 0.1 s	> 1 s	> 5 s	fin.	CPU	solved
without merging rows	1.02	1.32	1.22	1.23	362	1.06	214
explicit tableau	1.00	1.14	1.14	1.11	356	1.03	218
without long-step ratio test	1.01	1.03	1.03	1.05	362	1.01	218
farthest-bound order	1.00	1.01	1.00	1.01	362	1.06	218
warm start of the other program	1.01	1.01	0.99	0.99	364	1.04	218
without devex	1.00	1.00	1.00	1.00	363	1.00	217
without the limit of 60 pivots	1.00	1.01	1.01	1.01	364	1.01	217

5.5 Ablation

Table 5 changes one component of SC at a time, inside the search, with the affine relaxation. The search trees change by less than 3% in every row. Four components carry the time. Merging proportional rows is the largest: without it SC is 22–32% slower on systems of equations, with three fewer runs finished, and 6–7% slower on optimization problems, with three fewer problems solved; the twin rows of each equation double m, and on Geneig-icse the run takes five times longer. The engine: with the explicit tableau of Section 6 instead of the revised engine, SC is 11–14% slower on systems of equations, with nine fewer runs finished, and 3–4% slower on optimization problems. The long-step ratio test saves 3–5% on systems of equations and is neutral on optimization problems, whose LPs are shorter. And on optimization problems the order and the warm start: the farthest bound first is 5–6% slower and the dual warm start 4%; on systems of equations both choices cost the same. Devex and the limit of 60 pivots per LP change the time by less than 1%: with the affine relaxation, these rules act on few LPs.

6 Discussion

Why SC wins. SC does not contract with a better polytope: all engines solve the same relaxation and feed the same certificate. Its advantage has two sources. The first is cost. On the same LPs a pass of SC costs 0.55–0.62 times that of SoPlex and 0.73–0.82 times that of its simplex driven directly (Table 2), and SC does not pivot less: it makes 28–59% more pivots per LP than SoPlex. What it saves is the work that the certificate makes unnecessary, a refactorization and a verification at every LP, together with the setup of a call and the phase 1 that a change of objective or of the box would otherwise require (Section 5.2). About half of the gap with SoPlex as used by Ibex is its interface, and a careful integration of its simplex recovers it; the other half is in the simplex itself. The second source is the certificate at every basis: where the black box ends without an optimum the bound is lost, while the multipliers of SC still certify it. With the same iteration limit in every engine this is rare (Section 5.3), and per node the engines contract alike; along the search, small differences in either direction compound, and on Geneig-icse SC ends with 0.80 of the cells of SoPlex. The gain of SC is therefore mostly cost: on Geneig-icse a pass of SC costs 0.39 times

that of SoPlex on the same boxes.

Where it loses. Both sources are worth most when LPs are short. A pass removes a fixed cost per call, so its weight falls as the pivots per LP grow, and from 15–30 rows the pivots dominate (Section 5.4). There SC needs more of them than SoPlex, for three reasons that multiply: the vertex where it stops inside the optimal face of a degenerate polytope is a worse start for the next bound, its pivoting rules need more pivots from the same basis, and its pivots cost more, because the dual ratio test scans the whole pivot row and x_B is recomputed instead of updated. The first is the most specific to this setting and the one we could not fix: on optimization problems SoPlex ends 13–40% of its LPs at a vertex that is already optimal for the next bound, and started from those bases SC needs as few pivots, but neither a tie-break toward the corners of the box inside the optimal face nor a short walk inside that face (up to ten primal steps that keep the optimum) made SC end at such vertices. The second and third are engineering, and a revised engine with the pricing and the updates of SoPlex would remove them. Until then, a contractor can choose the engine per node. Solving the pass with SC when the relaxation of the previous node had at most 30 rows, and with the bare simplex otherwise, stays within 2% of the better engine in every configuration of Table 3. With `compo` on systems of equations this hybrid is 8–11% faster than SoPlex and the best configuration of the solver, exploring 0.47–0.56 of the cells of SC with the affine relaxation in 0.86–0.97 of the time; on optimization problems the best remains SC with the affine relaxation. A lower threshold fails: at 15 rows the hybrid hands Geneig-icse to the bare simplex, which loses bounds that SC certifies, and the run takes 2.2 times longer.

What did not pay. We also measured on the whole benchmark several ideas that use the certificate or the tableau beyond the pass, and none paid. Reusing certificates of an ancestor box keeps them valid, by Proposition 3, but not strong: tested on the slices of ACID, the multipliers of the node itself almost never prune a slice, and those of an ancestor prune slices that the linear contractor of the node prunes anyway. Truncating each LP when its anytime bound (Proposition 5) comes within a tolerance of what the LP can still gain saves pivots ($\times$0.84) but weakens the contraction (cells $\times$1.39–1.60); only the fixed limit of 60 pivots, which acts on a few long LPs, pays. Limiting only the first LP of each pass, the longest, saves its pivots without moving them to the next LP, but the first LP is too small a part of a node for this to matter. Propagating with HC4 in the middle of a pass, when a bound has just contracted by more than half, empties the box often, but those boxes would be emptied by the rest of the pass anyway. Cuts at the LP optima reduce the cells ($\times$0.80–0.88) at a higher cost (CPU $\times$1.18–1.31). A Lagrangian bound improved by monotonicity tightens the LP bound in most applications, but costs more than it saves (CPU $\times$1.08 to $\times$3.5). Inheriting the basis of the parent node costs only a factorization with the revised engine and cuts the pivots of the first LP of a node by a factor of three or more, but the first LP is a small part of the node: the time changes by less than 4% on optimization problems and grows by 1–5% on systems of equations.

Three components of earlier versions of SC were dropped for the same reason. Keeping the whole tableau $T = B^{-1}\bar{A}$ explicitly, row by row in sparse form, makes a pivot a Gauss–Jordan elimination with no linear system to solve; our first versions used it on relaxations of up to 30 rows, but with the other components in place it is 11–14% slower than the revised engine on systems of equations (Table 5), and 1–2% slower even when limited to 30 rows. With the tableau at hand, the bounds can also be ordered by the number of dual infeasibilities that the next LP must repair, which can be counted without pivoting; this saved up to 2% with the tableau, and the nearest bound first does as well with the revised engine. Finally, the bound problems are highly dual degenerate:

with z_k basic in row r, every column that does not meet row r has a zero reduced cost, and on some systems (Virasoro, Katsura) 80–95% of the dual pivots were degenerate. Perturbing the zero reduced costs after relocation, the standard remedy [18], saved 3–4% on systems of equations in earlier versions; with the other components it makes SC 1–2% slower. The pattern is that doing more work inside a node does not change the search tree; what changed it was what is computed: the merged relaxation, the stability filter of the ratio test, the Farkas ray of the infeasible row, the tolerance.

Limits. SC removes the external LP solver from the contractor, not from the optimizer: the upper-bounding of IbexOpt still solves its LPs with SoPlex, with every engine. On problems with very narrow equalities the upper-bounding becomes the bottleneck: on bearing, whose eight equalities are relaxed to $|h| \leq 10^{-8}$, the lower bound of SC is good but feasible points are found only in boxes of width 10^{-4} to 10^{-3}, and the search tree changes by two orders of magnitude between seeds; we excluded it, with alkyl and alkylbis, from the means. Relaxations of several hundred rows are still expensive for every engine: the systems of 400 to 1 000 variables of the benchmark exceed the limits of 60 s and 6 GB with all of them. Finally, the gain of SC is one of small relaxations: on the relaxations that make the best solver of systems of equations, SC alone is slower than the bare simplex of SoPlex, and the gain there comes from the hybrid. One use of the certificate is left open: when the warm-started simplex declares an infeasibility that the ray does not certify, SC solves the LP again from the slack basis (Section 4.2), but the multipliers of the basis where it stopped, moved along the ray as long as the bound improves, already give a certified bound without that second LP. Only the second LP can prove the box empty, and these cases are rare; we have not tried it.

7 Conclusion

A polytope-hull contractor solves $2n$ small, closely related LPs per node and needs a rigorous bound from each. We have shown that the certificate that makes the bound rigorous, the Neumaier–Shcherbina bound written in the homogeneous form of the relaxation, is also a design principle for the simplex that produces it. Since any basis certifies, the simplex needs no verification, no refactorization before declaring an optimum and no phase 1, can stop an LP at any basis, and solves the $2n$ problems on one shared basis. Inside the same contractor, with the same contraction, a pass of this simplex costs 0.55–0.62 times that of SoPlex, 0.73–0.82 times that of its simplex driven directly and 0.12–0.27 times that of HiGHS and Gurobi, and where a black box loses a bound the simplex still certifies it. With the affine relaxation the whole solver is 16–17% faster than with SoPlex on optimization problems and 8–9% on systems of equations. The gain belongs to small relaxations, where the fixed cost of a call dominates: beyond 15–30 rows SoPlex needs fewer pivots, mostly because the vertices where it ends an LP are better starting points for the next bound. A contractor that uses each engine where it is better is never worse than either, and is the best configuration on systems of equations. How to choose, inside the optimal face of a degenerate polytope, the vertex that serves the next bound is the open question.

References

[1] David L. Applegate, William Cook, Sanjeeb Dash, and Daniel G. Espinoza. Exact solutions to linear programming problems. *Operations Research Letters*, 35(6):693–699, 2007.

[2] Ignacio Araya, Frédéric Messine, Jordan Ninin, and Gilles Trombettoni. Hybridizing two linear relaxation techniques in interval-based solvers. *Journal of Global Optimization*, 91(3):437–456, 2025.

[3] Ignacio Araya, Gilles Trombettoni, and Bertrand Neveu. A contractor based on convex interval Taylor. In *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2012)*, Lecture Notes in Computer Science, pages 1–16, 2012.

[4] Ali Baharev, Tobias Achterberg, and Endre Rév. Computation of an extractive distillation column with affine arithmetic. *AIChE Journal*, 55(7):1695–1704, 2009.

[5] Pietro Belotti, Jon Lee, Leo Liberti, François Margot, and Andreas Wächter. Branching and bounds tightening techniques for non-convex MINLP. *Optimization Methods and Software*, 24(4-5):597–634, 2009.

[6] Frédéric Benhamou, Frédéric Goualard, Laurent Granvilliers, and Jean-François Puget. Revising hull and box consistency. In *Proceedings of the International Conference on Logic Programming (ICLP)*, pages 230–244, 1999.

[7] Gilles Chabert and Luc Jaulin. Contractor programming. *Artificial Intelligence*, 173(11):1079–1100, 2009.

[8] William Cook, Thorsten Koch, Daniel E. Steffy, and Kati Wolter. A hybrid branch-and-bound approach for exact rational mixed-integer programming. *Mathematical Programming Computation*, 5(3):305–344, 2013.

[9] Elizabeth D. Dolan and Jorge J. Moré. Benchmarking optimization software with performance profiles. *Mathematical Programming*, 91(2):201–213, 2002.

[10] Ferenc Domes and Arnold Neumaier. Rigorous filtering using linear relaxations. *Journal of Global Optimization*, 53(3):441–473, 2012.

[11] John J. Forrest and Donald Goldfarb. Steepest-edge simplex algorithms for linear programming. *Mathematical Programming*, 57(1-3):341–374, 1992.

[12] John J. H. Forrest and John A. Tomlin. Updated triangular factors of the basis to maintain sparsity in the product form simplex method. *Mathematical Programming*, 2:263–278, 1972.

[13] Ambros M. Gleixner, Timo Berthold, Benjamin Müller, and Stefan Weltge. Three enhancements for optimization-based bound tightening. *Journal of Global Optimization*, 67(4):731–757, 2017.

[14] Ambros M. Gleixner, Daniel E. Steffy, and Kati Wolter. Iterative refinement for linear programming. *INFORMS Journal on Computing*, 28(3):449–464, 2016.

[15] Paula M. J. Harris. Pivot selection methods of the Devex LP code. *Mathematical Programming*, 5(1):1–28, 1973.

[16] Christian Jansson. Rigorous lower and upper bounds in linear programming. *SIAM Journal on Optimization*, 14(3):914–935, 2004.

[17] Christian Keil and Christian Jansson. Computational experience with rigorous error bounds for the Netlib linear programming library. *Reliable Computing*, 12(4):303–321, 2006.

[18] Achim Koberstein. *The dual simplex method, techniques for a fast and stable implementation.* PhD thesis, Universität Paderborn, 2005. SIN VERIFICAR.

[19] Yahia Lebbah, Claude Michel, and Michel Rueher. A rigorous global filtering algorithm for quadratic constraints. *Constraints*, 10:47–65, 2005. SIN VERIFICAR (tomado de las referencias de Domes y Neumaier 2012).

[20] István Maros. A generalized dual phase-2 simplex algorithm. *European Journal of Operational Research*, 149(1):1–16, 2003.

[21] Arnold Neumaier. *Interval Methods for Systems of Equations.* Cambridge University Press, 1990.

[22] Arnold Neumaier. Complete search in continuous global optimization and constraint satisfaction. In *Acta Numerica 2004*, pages 271–369. Cambridge University Press, 2004. SIN VERIFICAR (tomado de las referencias de Domes y Neumaier 2012).

[23] Arnold Neumaier and Oleg Shcherbina. Safe bounds in linear and mixed-integer linear programming. *Mathematical Programming*, 99(2):283–296, 2004.

[24] Bertrand Neveu, Gilles Trombettoni, and Ignacio Araya. Adaptive constructive interval disjunction: algorithms and experiments. *Constraints*, 20(4):452–467, 2015.

[25] Jordan Ninin, Frédéric Messine, and Pierre Hansen. A reliable affine relaxation method for global optimization. *4OR*, 13(3):247–277, 2015.

[26] Victor Reyes and Ignacio Araya. Non-convex optimization: Using preconditioning matrices for optimally improving variable bounds in linear relaxations. *Mathematics*, 11(16):3549, 2023.

[27] Hong S. Ryoo and Nikolaos V. Sahinidis. A branch-and-reduce approach to global optimization. *Journal of Global Optimization*, 8(2):107–138, 1996.

[28] Roland Wunderling. *Paralleler und objektorientierter Simplex-Algorithmus.* PhD thesis, Technische Universität Berlin, 1996.

[29] Kiyotaka Yamamura and T. Fujioka. Finding all solutions of systems of nonlinear equations using the dual simplex method. In *Proc. 2001 Int. Symp. Nonlinear Theory and Its Applications (NOLTA)*, pages 219–222, 2001. SIN VERIFICAR (tomado de las referencias de Yamamura y Tanaka 2006).

[30] Kiyotaka Yamamura and Katsumasa Tanaka. Finding all solutions of weakly nonlinear equations using the dual simplex method. *Electronics and Communications in Japan (Part III: Fundamental Electronic Science)*, 89(7):1–7, 2006.